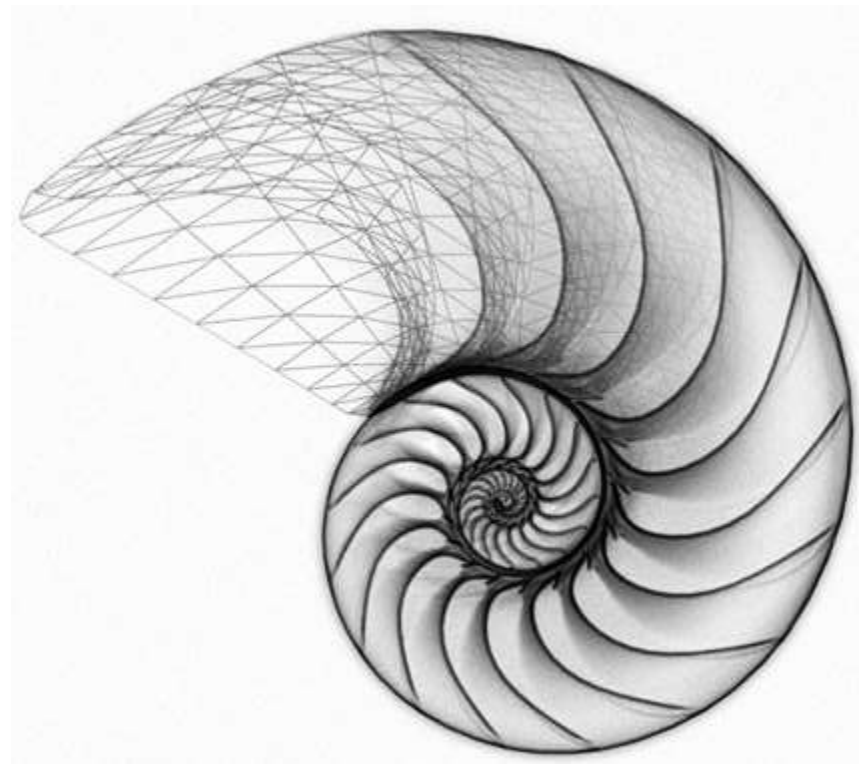




Lecture 14

Heaps and Priority Queues

CSE225: Data Structures and Algorithms

Priority Queues

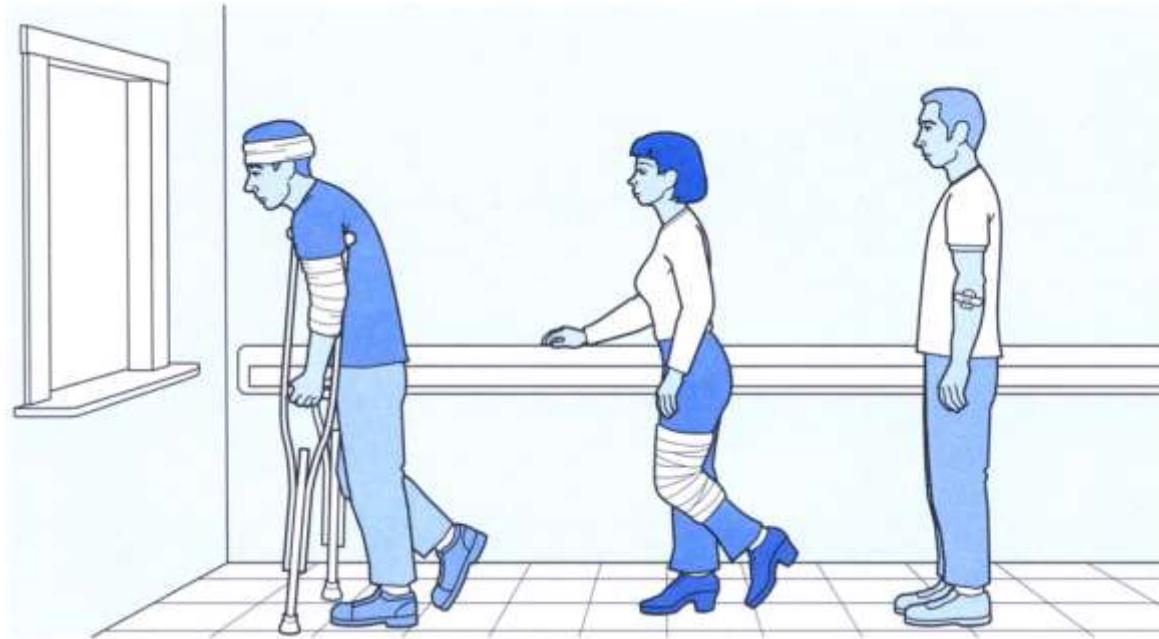
- Items can be added and deleted (like queues)
 - Not necessarily maintains the First In First Out order

Priority Queues

- Items can be added and deleted (like queues)
 - Not necessarily maintains the First In First Out order
 - Items can be added in any order
 - Always, the item with the highest priority is deleted

Priority Queues

- Items can be added and deleted (like queues)
 - Not necessarily maintains the First In First Out order
 - Items can be added in any order
 - Always, the item with the highest priority is deleted



Priority Queue Specification

Structure:	The priority queue is arranged to support access to the highest-priority item.
Operations:	
MakeEmpty	
Function	Initializes the queue to an empty state.
Postcondition	Queue is empty.
Boolean IsEmpty	
Function	Tests whether the queue is empty.
Postcondition	Returns true if queue is empty and false otherwise.
Boolean IsFull	
Function	Tests whether the queue is full.
Postcondition	Returns true if queue is full and false otherwise.

Priority Queue Specification

Enqueue(ItemType newItem)

Function	Adds newItem to the queue.
----------	----------------------------

Postcondition	If (the priority queue is full), exception FullPQ is thrown; else newItem is in the queue.
---------------	---

Dequeue(ItemType& item)

Function	Removes element with highest priority and returns it in item.
----------	---

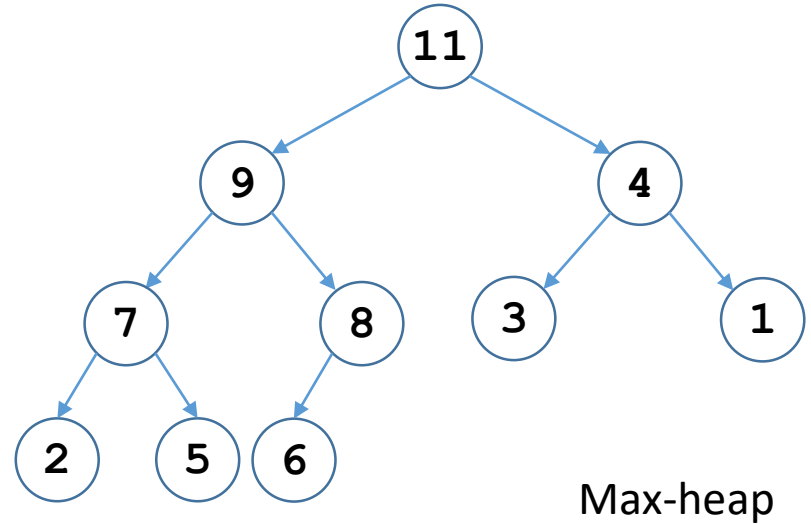
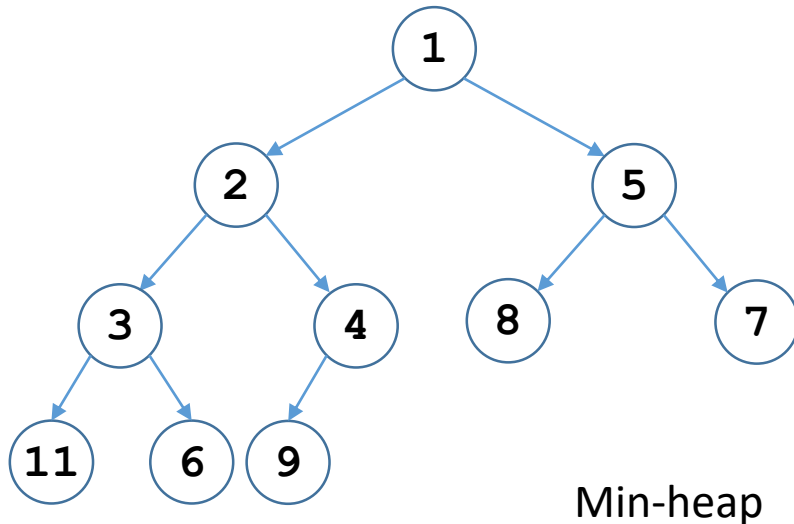
Postcondition	If (the priority queue is empty), exception EmptyPQ is thrown; else highest priority element has been removed from queue. item is a copy of removed element.
---------------	--

Heaps

- A complete binary tree
 - Each of the elements contains a value that is less than or equal to the value of each of its children (Min-heap)
 - Each of the elements contains a value that is greater than or equal to the value of each of its children (Max-heap)

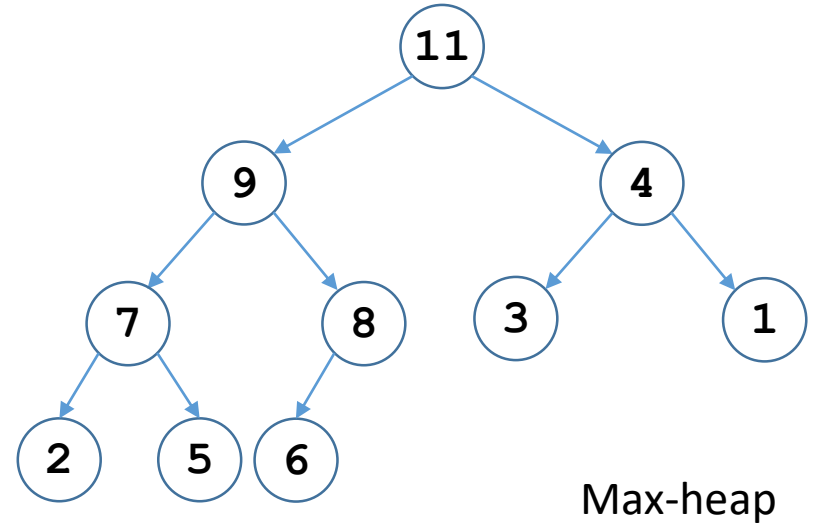
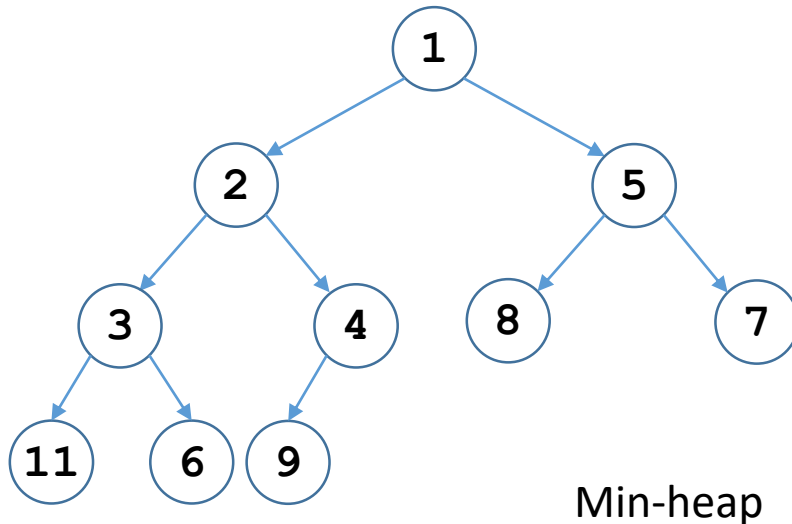
Heaps

- A complete binary tree
 - Each of the elements contains a value that is less than or equal to the value of each of its children (Min-heap)
 - Each of the elements contains a value that is greater than or equal to the value of each of its children (Max-heap)



Heaps

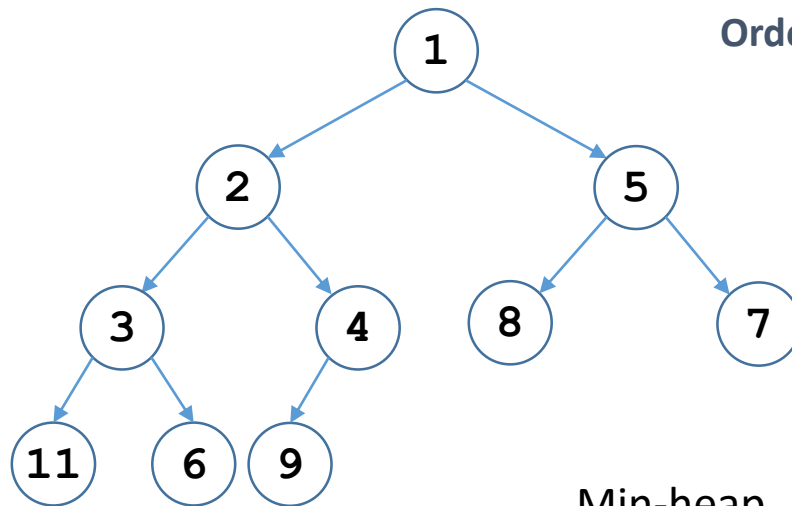
- A complete binary tree ← **Shape property**
 - Each of the elements contains a value that is less than or equal to the value of each of its children (Min-heap)
 - Each of the elements contains a value that is greater than or equal to the value of each of its children (Max-heap)



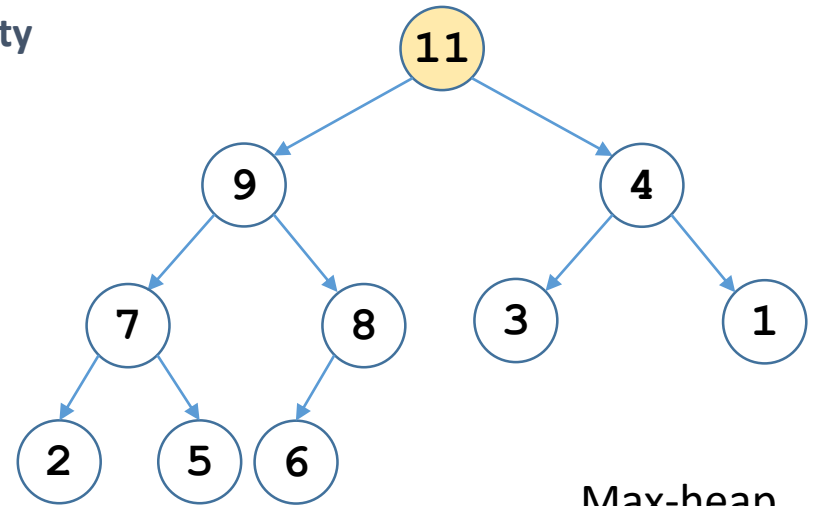
- The shape of all heaps with a given number of elements is the same.

Heaps

- A complete binary tree Shape property
 - Each of the elements contains a value that is less than or equal to the value of each of its children (Min-heap)
 - Each of the elements contains a value that is greater than or equal to the value of each of its children (Max-heap)



Min-heap

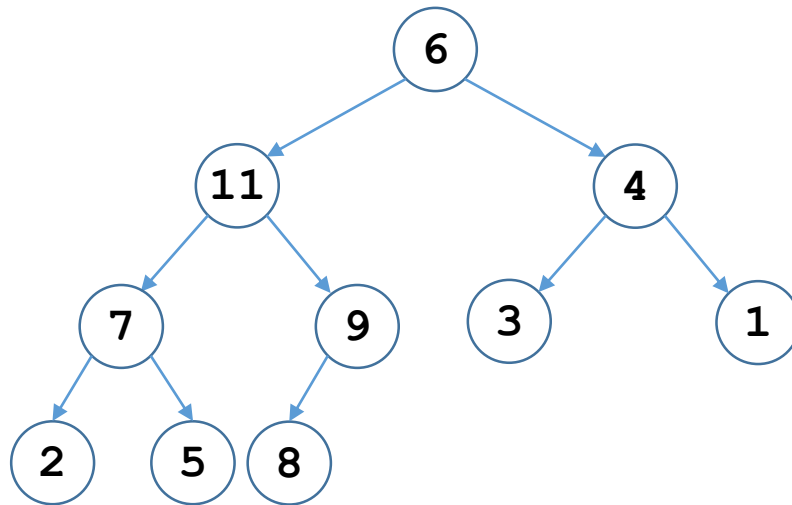


Max-heap

- The shape of all heaps with a given number of elements is the same.
- The root node always contains the largest value in the heap (in addition, the subtrees are heaps as well).

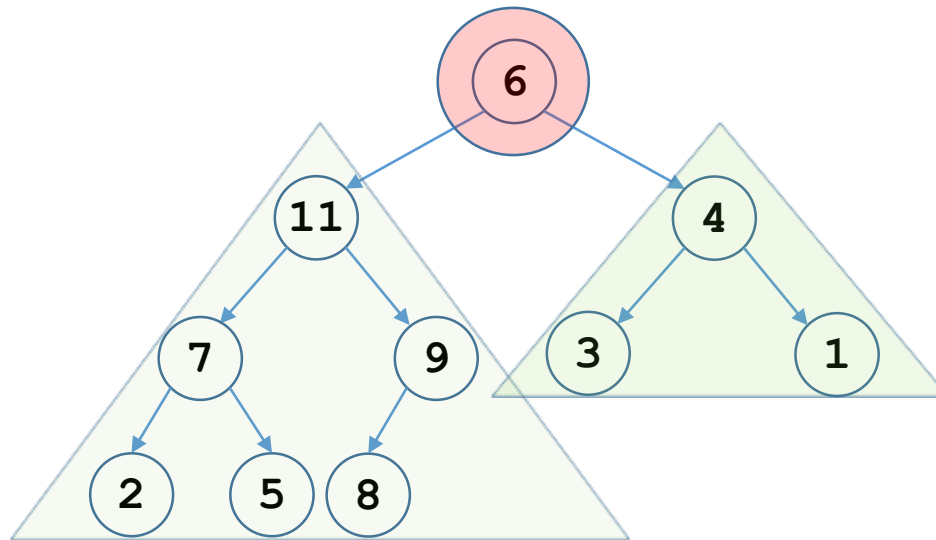
The **ReheapDown** operation

- Suppose that, the order property is violated by the **root node** only (not any other node, they are in place)
- Repair the structure so that it becomes a heap again (called **ReheapDown** operation)
 - move the element down from the root position until it ends up in a position where the order property is satisfied



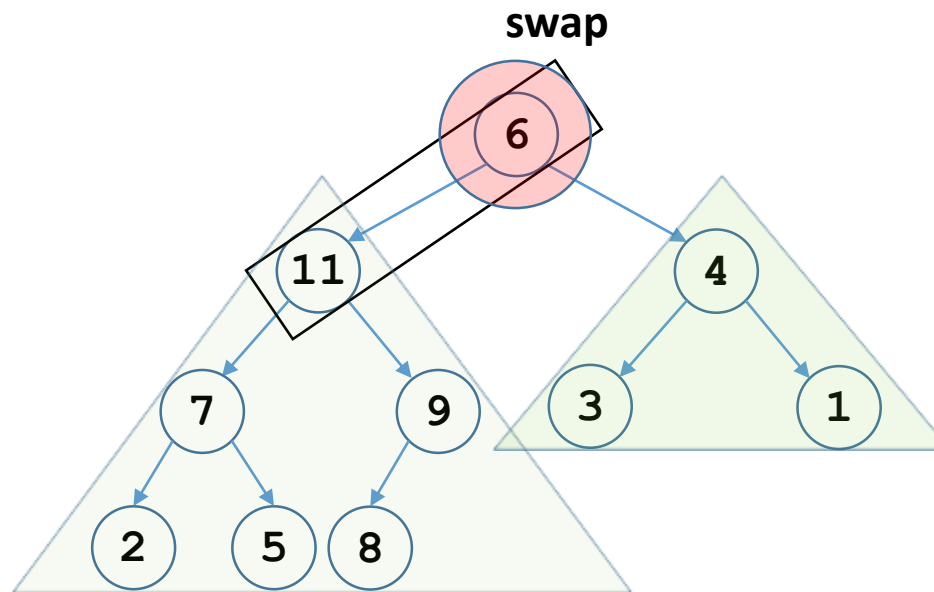
The **ReheapDown** operation

- Suppose that, the order property is violated by the **root node** only (not any other node, they are in place)
- Repair the structure so that it becomes a heap again (called **ReheapDown** operation)
 - move the element down from the root position until it ends up in a position where the order property is satisfied



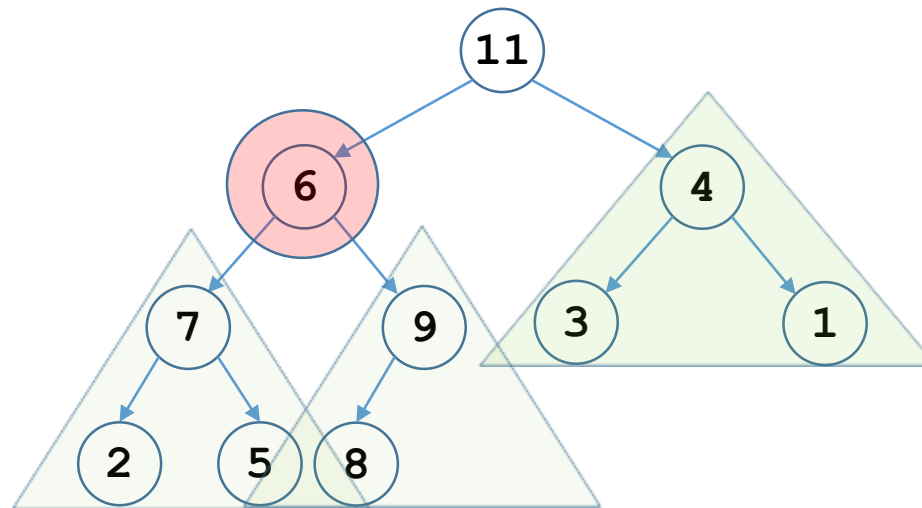
The **ReheapDown** operation

- Suppose that, the order property is violated by the **root node** only (not any other node, they are in place)
- Repair the structure so that it becomes a heap again (called **ReheapDown** operation)
 - move the element down from the root position until it ends up in a position where the order property is satisfied



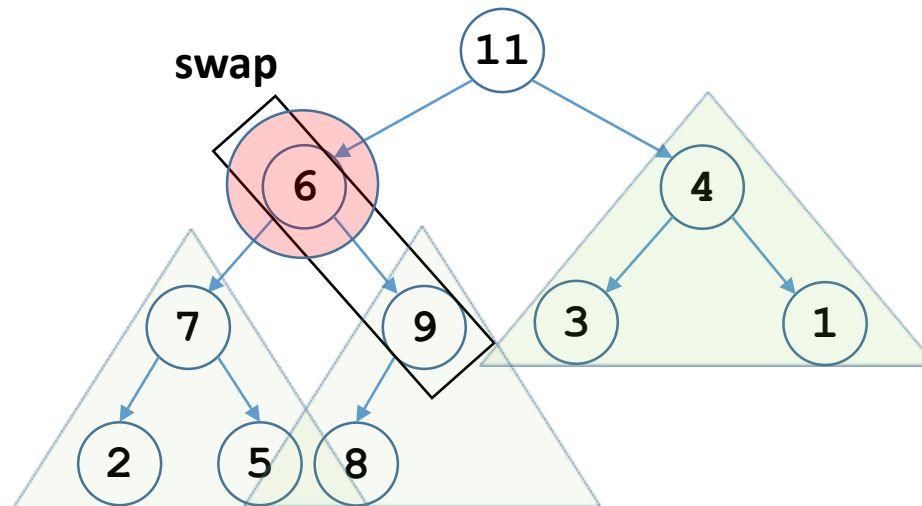
The **ReheapDown** operation

- Suppose that, the order property is violated by the **root node** only (not any other node, they are in place)
- Repair the structure so that it becomes a heap again (called **ReheapDown** operation)
 - move the element down from the root position until it ends up in a position where the order property is satisfied



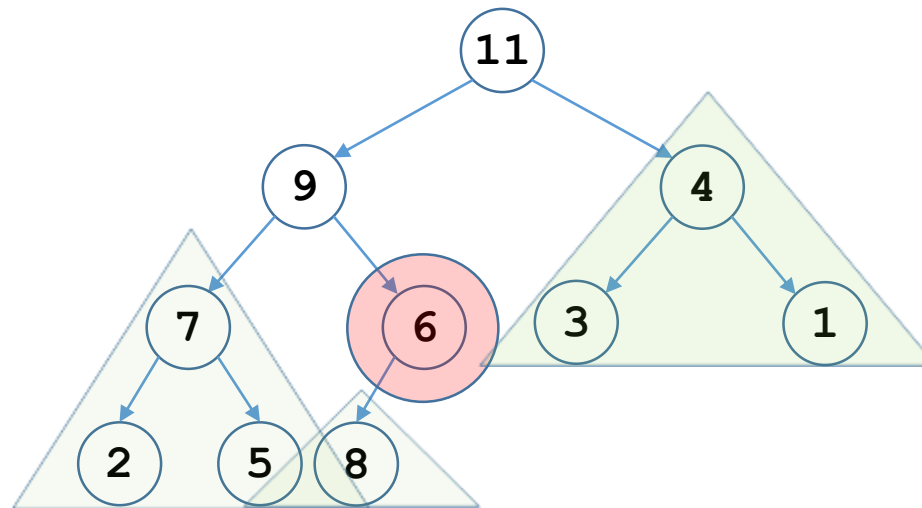
The **ReheapDown** operation

- Suppose that, the order property is violated by the **root node** only (not any other node, they are in place)
- Repair the structure so that it becomes a heap again (called **ReheapDown** operation)
 - move the element down from the root position until it ends up in a position where the order property is satisfied



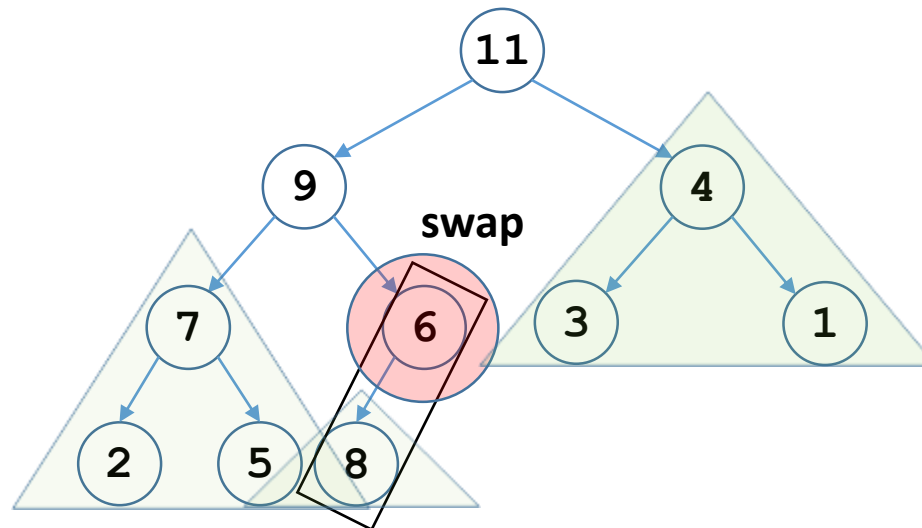
The **ReheapDown** operation

- Suppose that, the order property is violated by the **root node** only (not any other node, they are in place)
- Repair the structure so that it becomes a heap again (called **ReheapDown** operation)
 - move the element down from the root position until it ends up in a position where the order property is satisfied



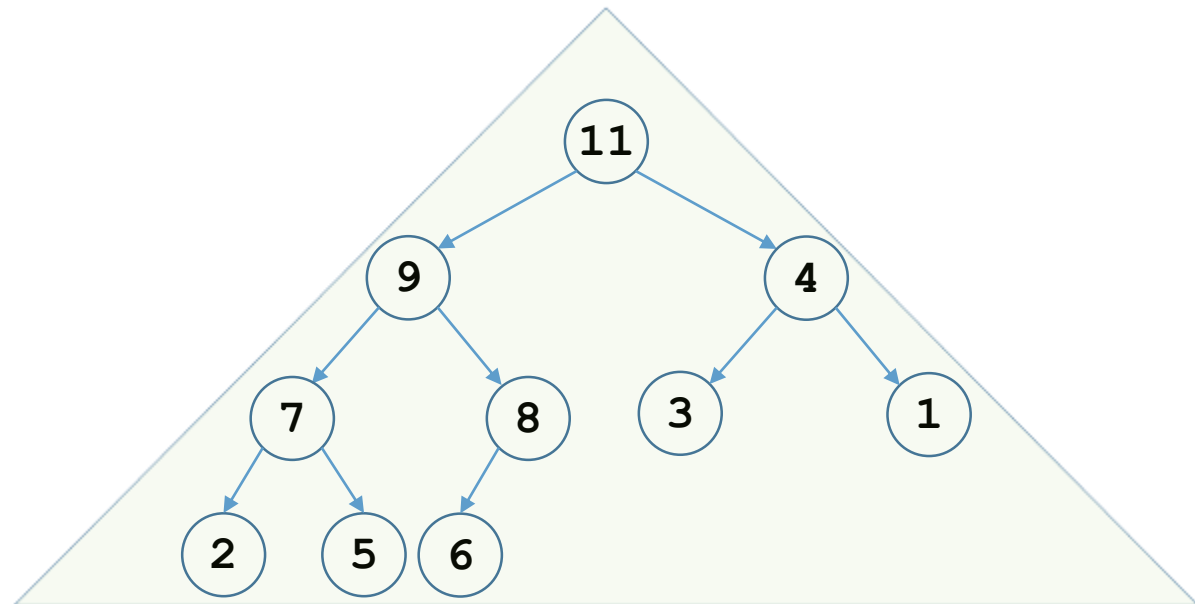
The **ReheapDown** operation

- Suppose that, the order property is violated by the **root node** only (not any other node, they are in place)
- Repair the structure so that it becomes a heap again (called **ReheapDown** operation)
 - move the element down from the root position until it ends up in a position where the order property is satisfied



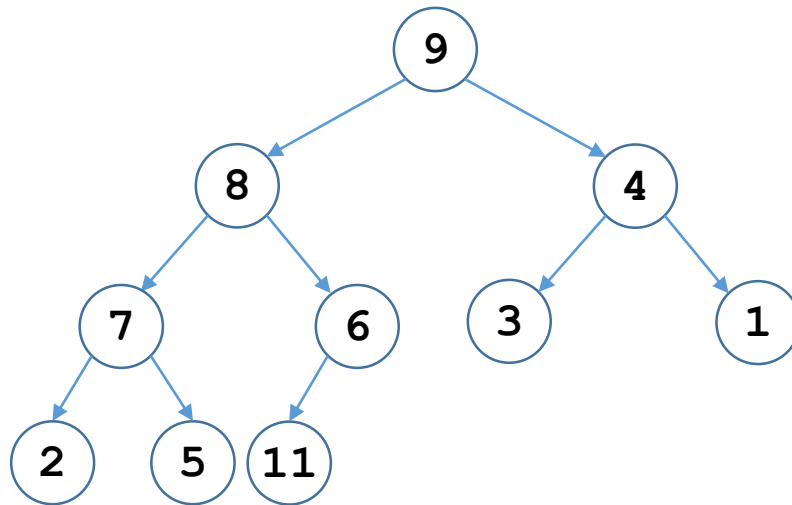
The **ReheapDown** operation

- Suppose that, the order property is violated by the **root node** only (not any other node, they are in place)
- Repair the structure so that it becomes a heap again (called **ReheapDown** operation)
 - move the element down from the root position until it ends up in a position where the order property is satisfied



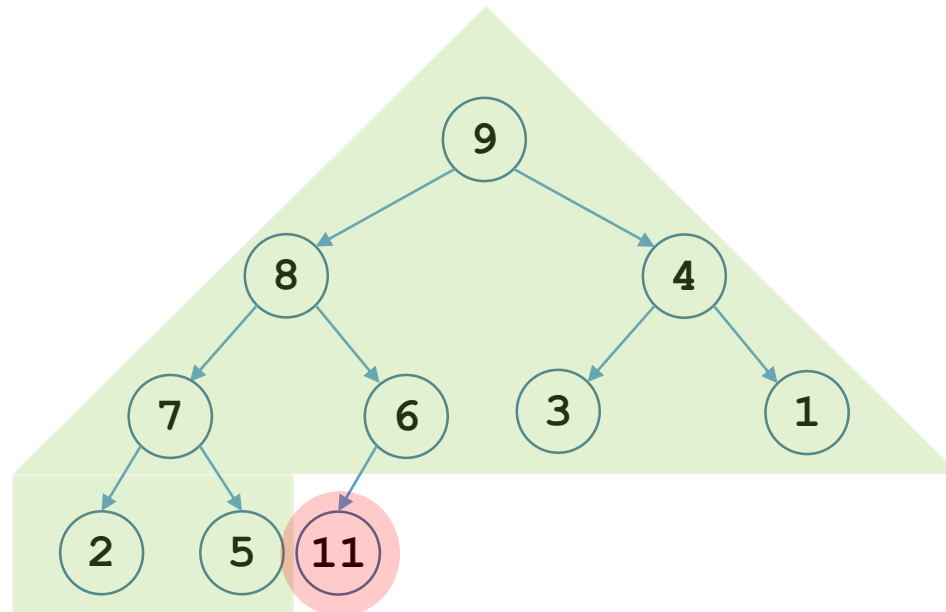
The **ReheapUp** operation

- Suppose that, the order property is violated by the **last leaf node** only (not any other node, they are in place)
- Repair the structure so that it becomes a heap again (called **ReheapUp** operation)
 - move the element up in the tree until it ends up in its correct position



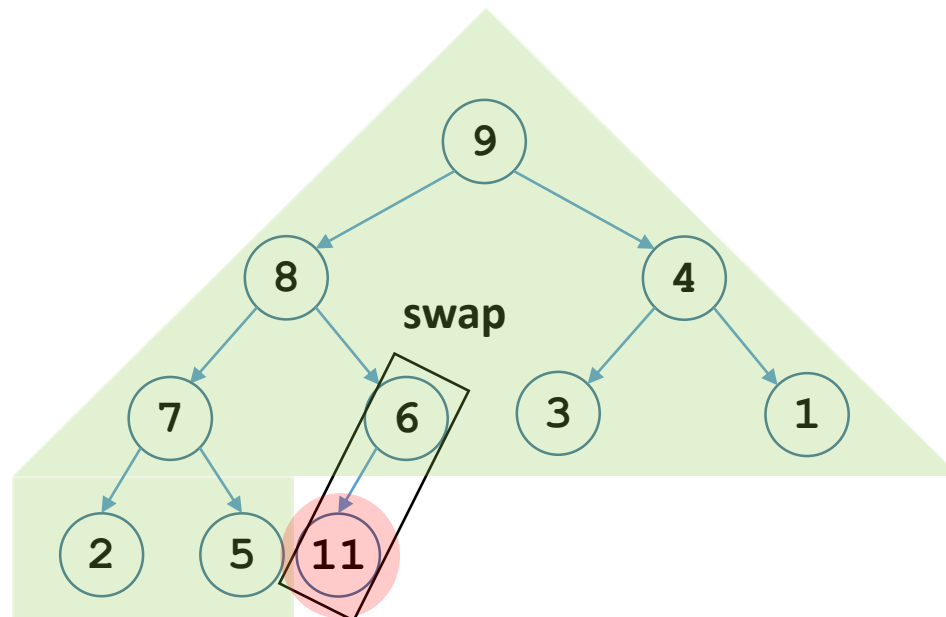
The **ReheapUp** operation

- Suppose that, the order property is violated by the **last leaf node** only (not any other node, they are in place)
- Repair the structure so that it becomes a heap again (called **ReheapUp** operation)
 - move the element up in the tree until it ends up in its correct position



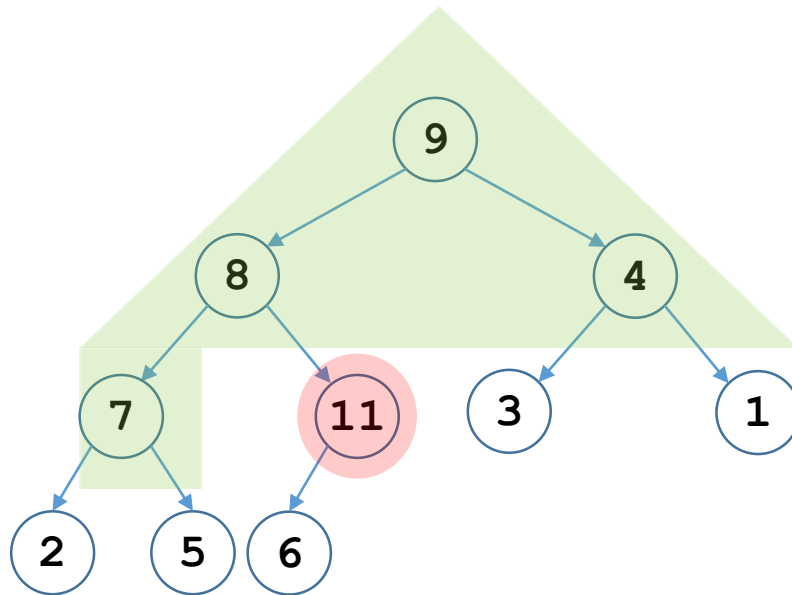
The **ReheapUp** operation

- Suppose that, the order property is violated by the **last leaf node** only (not any other node, they are in place)
- Repair the structure so that it becomes a heap again (called **ReheapUp** operation)
 - move the element up in the tree until it ends up in its correct position



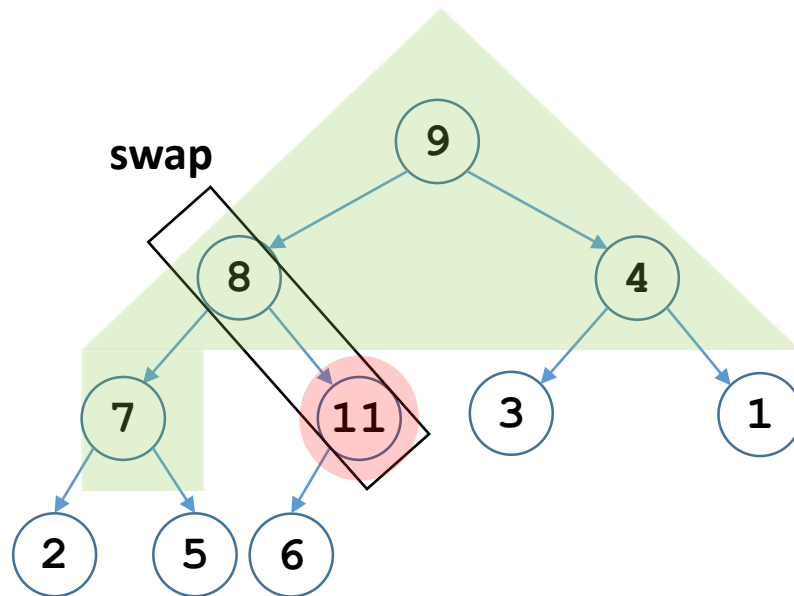
The **ReheapUp** operation

- Suppose that, the order property is violated by the **last leaf node** only (not any other node, they are in place)
- Repair the structure so that it becomes a heap again (called **ReheapUp** operation)
 - move the element up in the tree until it ends up in its correct position



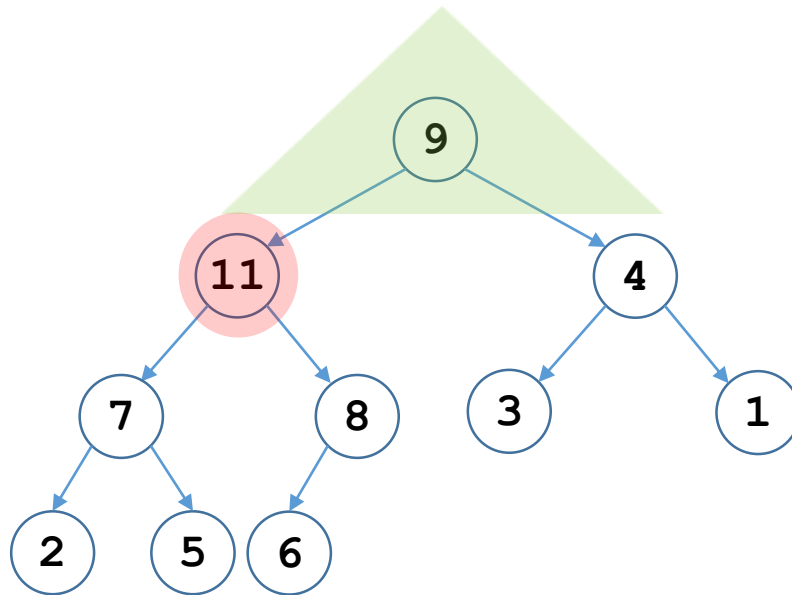
The **ReheapUp** operation

- Suppose that, the order property is violated by the **last leaf node** only (not any other node, they are in place)
- Repair the structure so that it becomes a heap again (called **ReheapUp** operation)
 - move the element up in the tree until it ends up in its correct position



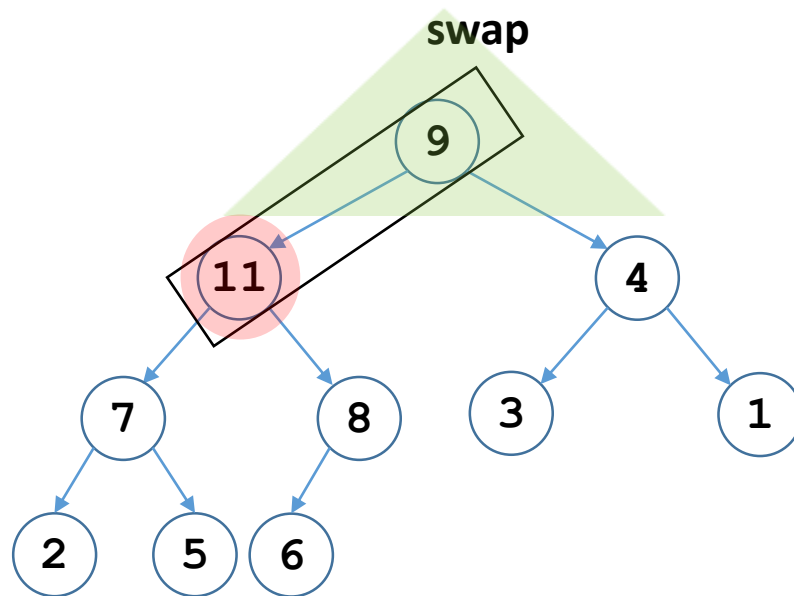
The **ReheapUp** operation

- Suppose that, the order property is violated by the **last leaf node** only (not any other node, they are in place)
- Repair the structure so that it becomes a heap again (called **ReheapUp** operation)
 - move the element up in the tree until it ends up in its correct position



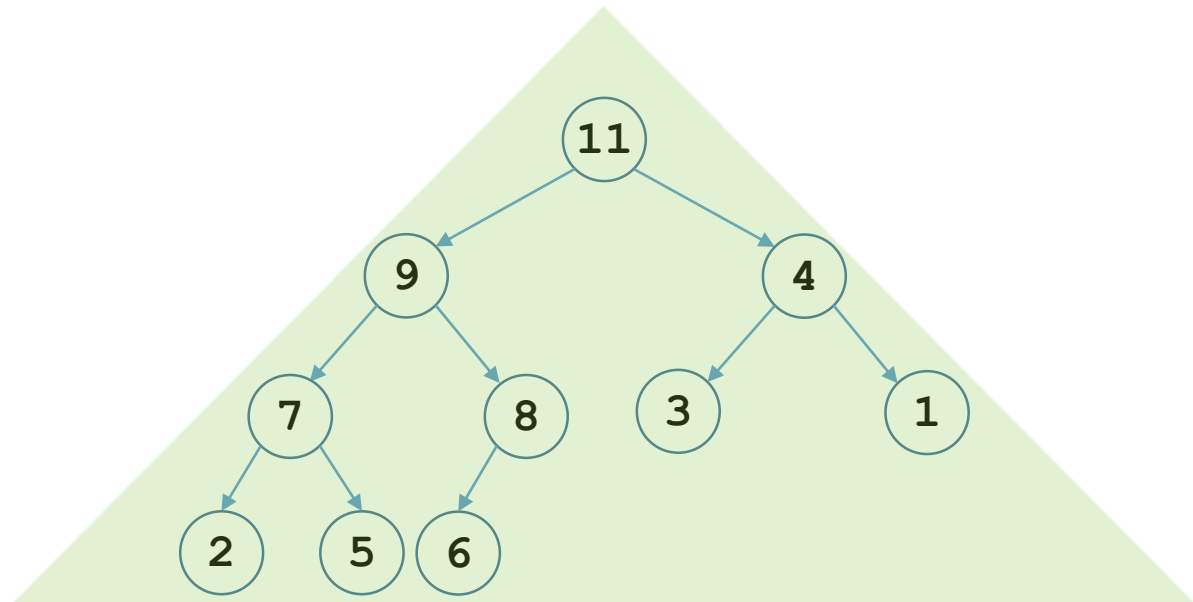
The **ReheapUp** operation

- Suppose that, the order property is violated by the **last leaf node** only (not any other node, they are in place)
- Repair the structure so that it becomes a heap again (called **ReheapUp** operation)
 - move the element up in the tree until it ends up in its correct position



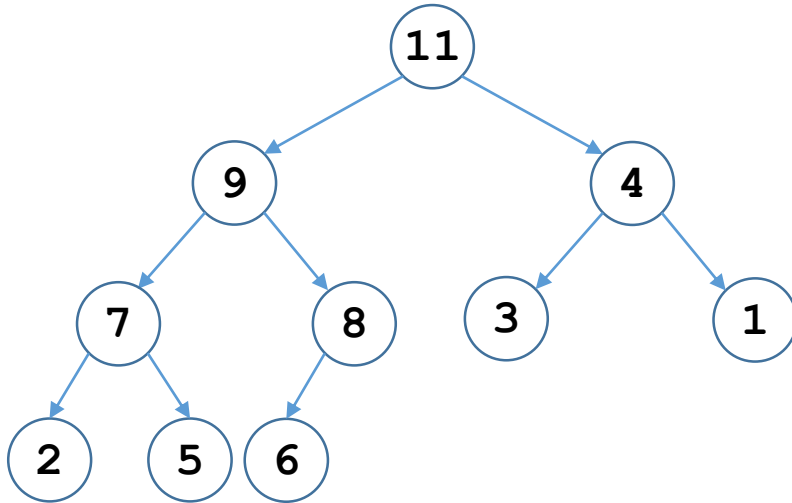
The **ReheapUp** operation

- Suppose that, the order property is violated by the **last leaf node** only (not any other node, they are in place)
- Repair the structure so that it becomes a heap again (called **ReheapUp** operation)
 - move the element up in the tree until it ends up in its correct position



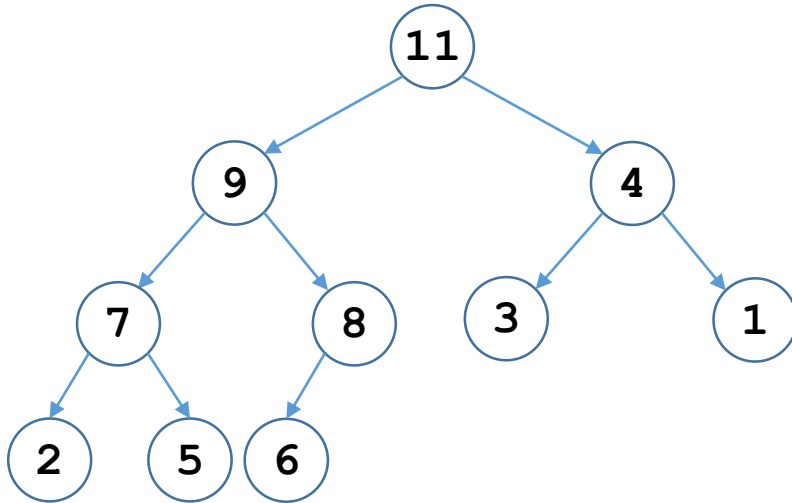
Heaps (Implementation Issue)

- We do not need linked structures to store a heap (since the tree is complete, there are not any “holes” in the tree)



Heaps (Implementation Issue)

- We do not need linked structures to store a heap (since the tree is complete, there are not any “holes” in the tree)

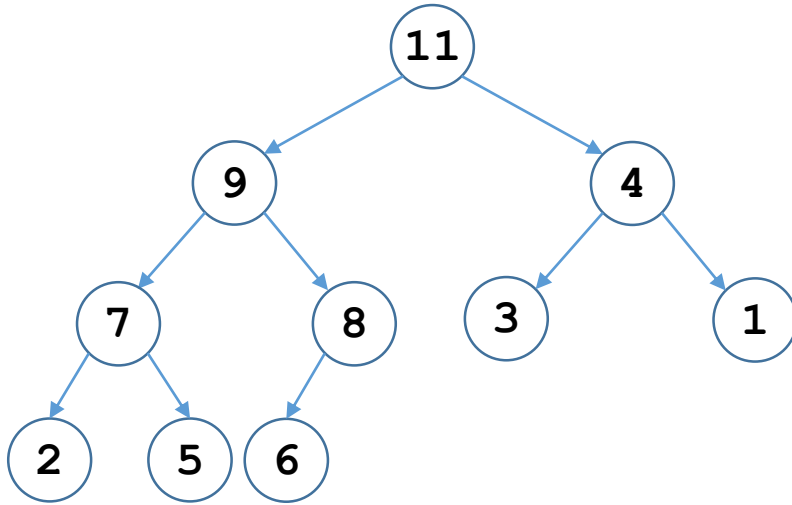


Index	Value
0	11
1	9
2	4
3	7
4	8
5	3
6	1
7	2
8	5
9	6

- We can arrays in order to store heaps.
 - Root at index 0.
 - For any value at index i ,
 - Left child at index $2i+1$
 - Right child at index $2i+2$
 - For any value at index i
 - Parent is at index $(i-1)/2$

Heaps (Implementation Issue)

- We do not need linked structures to store a heap (since the tree is complete, there are not any “holes” in the tree)



```
template<class ItemType>
struct HeapType
{
    void ReheapDown(int root, int bottom);
    void ReheapUp(int root, int bottom);
    ItemType* elements;
    int numElements;
};
```

Index	Value
0	11
1	9
2	4
3	7
4	8
5	3
6	1
7	2
8	5
9	6

heap.cpp

```
template <class ItemType>
void Swap(ItemType& one, ItemType& two)
{
    ItemType temp;
    temp = one;
    one = two;
    two = temp;
}

template<class ItemType>
void HeapType<ItemType>::ReheapUp(int root, int bottom)
{
    int parent;

    if (bottom > root)
    {
        parent = (bottom-1) / 2;
        if (elements[parent] < elements[bottom])
        {
            Swap(elements[parent], elements[bottom]);
            ReheapUp(root, parent);
        }
    }
}
```

heap.cpp

```
template <class ItemType>
void Swap(ItemType& one, ItemType& two)
{
    ItemType temp;
    temp = one;
    one = two;
    two = temp;
}

template<class ItemType>
void HeapType<ItemType>::ReheapUp(int root, int bottom)
{
    int parent;

    if (bottom > root)
    {
        parent = (bottom-1) / 2;
        if (elements[parent] < elements[bottom])
        {
            Swap(elements[parent], elements[bottom]);
            ReheapUp(root, parent);
        }
    }
}
```

$O(1)$

$O(\log N)$

heap.cpp

```
template<class ItemType>
void HeapType<ItemType>::ReheapDown(int root, int bottom)
{
    int maxChild, rightChild = root*2+2, leftChild = root*2+1;
    if (leftChild <= bottom) //there is at least one child
    {
        if (leftChild == bottom) //it is the only child
            maxChild = leftChild;
        else //there are two children
        {
            if (elements[leftChild] <= elements[rightChild])
                maxChild = rightChild;
            else
                maxChild = leftChild;
        }
        if (elements[root] < elements[maxChild])
        {
            Swap(elements[root], elements[maxChild]);
            ReheapDown(maxChild, bottom);
        }
    }
}
```


heap.cpp

```
template<class ItemType>
void HeapType<ItemType>::ReheapDown(int root, int bottom)
{
    int maxChild, rightChild = root*2+2, leftChild = root*2+1;
    if (leftChild <= bottom) //there is at least one child
    {
        if (leftChild == bottom) //it is the only child
            maxChild = leftChild;
        else //there are two children
        {
            if (elements[leftChild] <= elements[rightChild])
                maxChild = rightChild;
            else
                maxChild = leftChild;
        }
        if (elements[root] < elements[maxChild])
        {
            Swap(elements[root], elements[maxChild]);
            ReheapDown(maxChild, bottom);
        }
    }
}
```

$O(\log N)$

pqtype.h

```
#ifndef PQTYPE_H_INCLUDED
#define PQTYPE_H_INCLUDED

class FullPQ{};
class EmptyPQ{};
#include "heap.h"
template<class ItemType>
class PQType
{
public:
    PQType(int);
    ~PQType();
    void MakeEmpty();
    bool IsEmpty();
    bool IsFull();
    void Enqueue(ItemType newItem);
    void Dequeue(ItemType& item);
private:
    int length;
    HeapType<ItemType> items;
    int maxItems;
};
#endif // PQTYPE_H_INCLUDED
```

pqtype.cpp

```
template<class ItemType>
PQType<ItemType>::PQType(int max)
{
    maxItems = max;
    items.elements = new ItemType[max];
    length = 0;
}
```

```
template<class ItemType>
PQType<ItemType>::~~PQType()
{
    delete [] items.elements;
}
```

```
template<class ItemType>
void PQType<ItemType>::MakeEmpty()
{
    length = 0;
}
```

pqtype.cpp

```
template<class ItemType>
bool PQType<ItemType>::IsFull()
{
    return length == maxItems;
}

template<class ItemType>
bool PQType<ItemType>::IsEmpty()
{
    return length == 0;
}
```

pqtype.cpp

```
template<class ItemType>
PQType<ItemType>::PQType(int max)
{
    maxItems = max;
    items.elements = new ItemType[max];
    length = 0;
}
```

O(1)

```
template<class ItemType>
PQType<ItemType>::~~PQType()
{
    delete [] items.elements;
}
```

O(1)

```
template<class ItemType>
void PQType<ItemType>::MakeEmpty()
{
    length = 0;
}
```

O(1)

pqtype.cpp

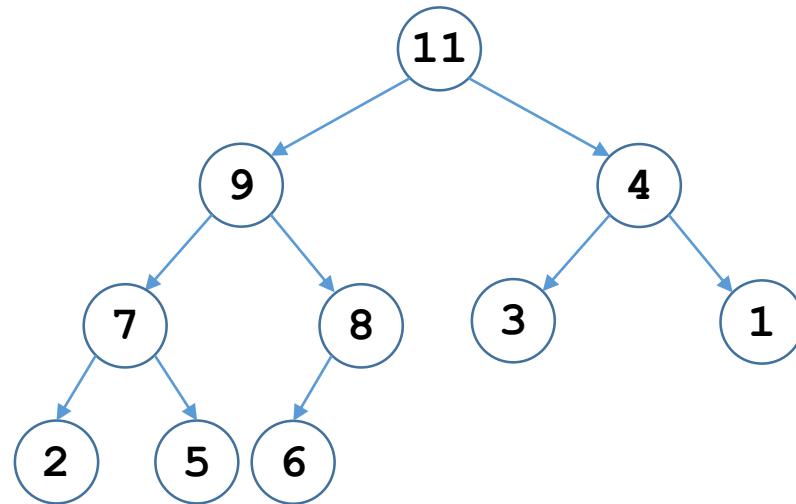
```
template<class ItemType>
bool PQType<ItemType>::IsFull()
{
    return length == maxItems;
}
```

O(1)

```
template<class ItemType>
bool PQType<ItemType>::IsEmpty()
{
    return length == 0;
}
```

O(1)

The **Enqueue** operation



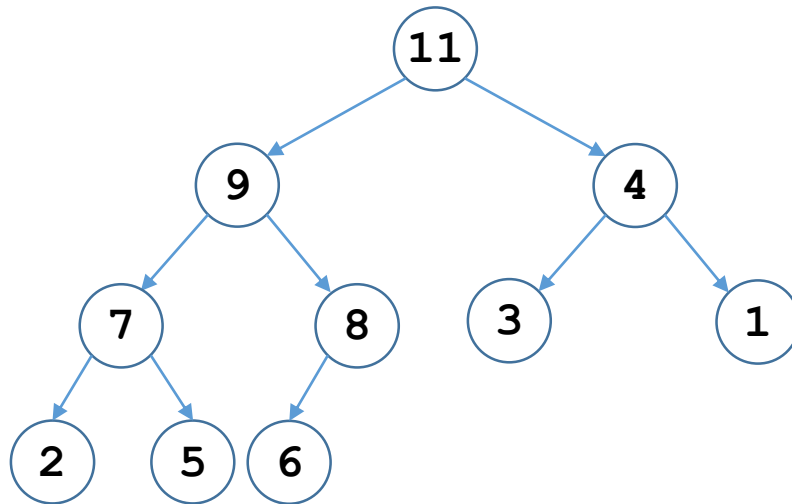
Enqueue 15

Index	0	1	2	3	4	5	6	7	8	9	10
value	11	9	4	7	8	3	1	2	5	6	?

Length = 10

The **Enqueue** operation

- Increment **length**



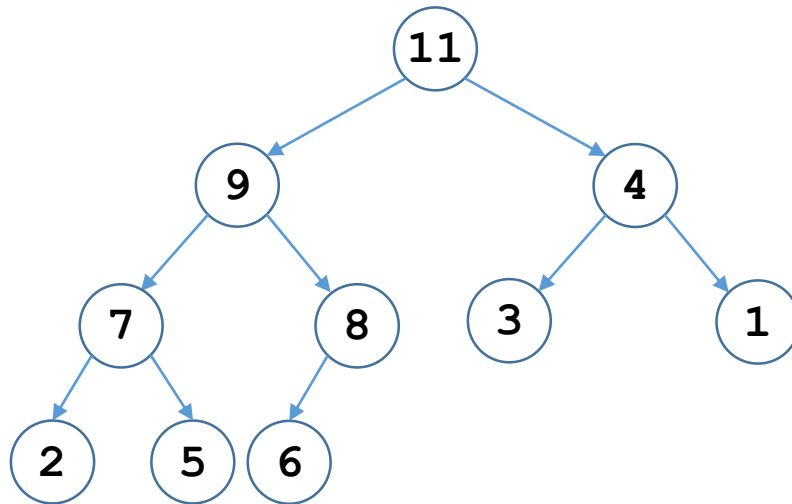
Enqueue 15

Index	0	1	2	3	4	5	6	7	8	9	10
value	11	9	4	7	8	3	1	2	5	6	?

Length = 10

The **Enqueue** operation

- Increment **length**



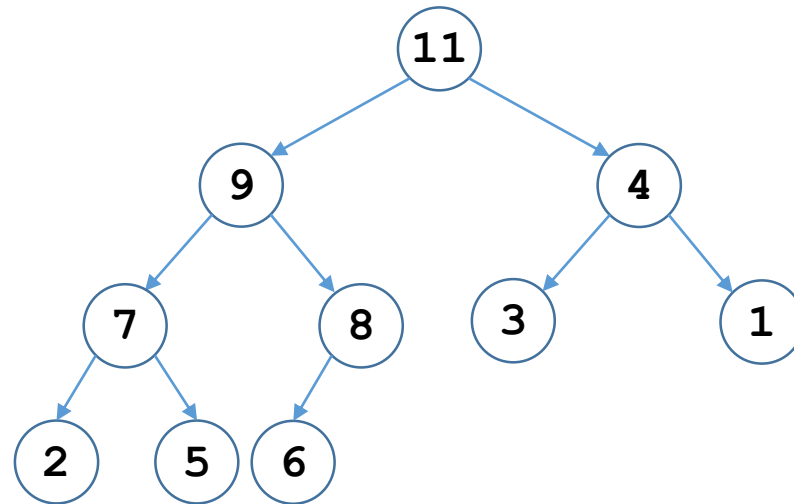
Enqueue 15

Index	0	1	2	3	4	5	6	7	8	9	10
value	11	9	4	7	8	3	1	2	5	6	?

Length = 11

The **Enqueue** operation

- Increment **length**
- Add the item to be enqueued as the last leaf node (at index **length-1**)



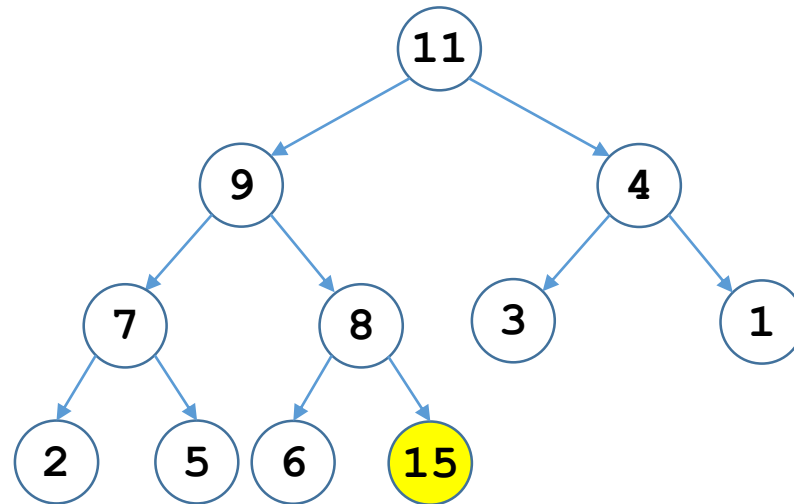
Enqueue 15

Index	0	1	2	3	4	5	6	7	8	9	10
value	11	9	4	7	8	3	1	2	5	6	?

Length = 11

The **Enqueue** operation

- Increment **length**
- Add the item to be enqueued as the last leaf node (at index **length-1**)



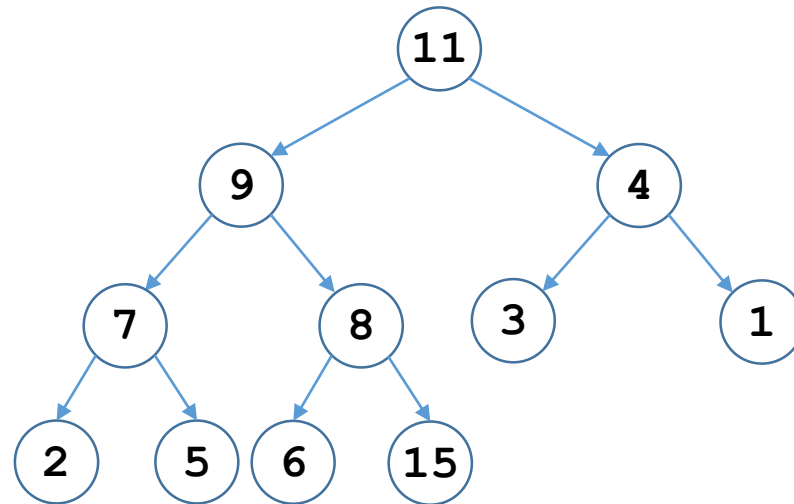
Enqueue 15

Index	0	1	2	3	4	5	6	7	8	9	10
value	11	9	4	7	8	3	1	2	5	6	15

Length = 11

The **Enqueue** operation

- Increment **length**
- Add the item to be enqueued as the last leaf node (at index **length-1**)
- Perform **ReheapUp** operation



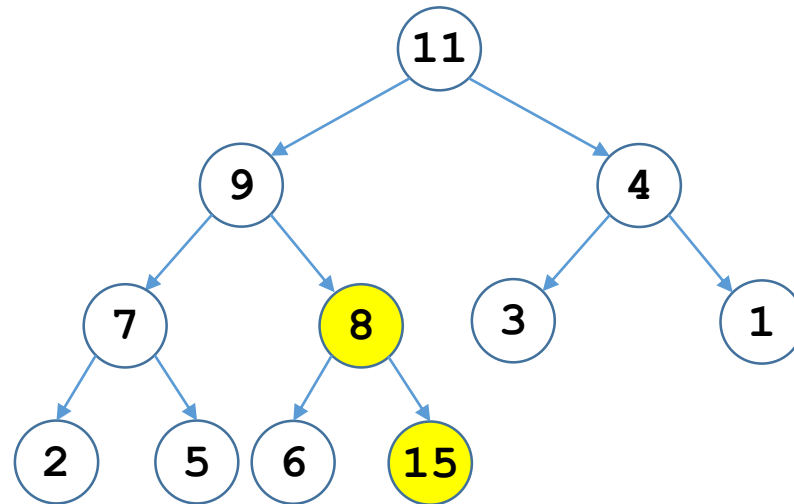
Enqueue 15

Index	0	1	2	3	4	5	6	7	8	9	10
value	11	9	4	7	8	3	1	2	5	6	15

Length = 11

The **Enqueue** operation

- Increment **length**
- Add the item to be enqueued as the last leaf node (at index **length-1**)
- Perform **ReheapUp** operation



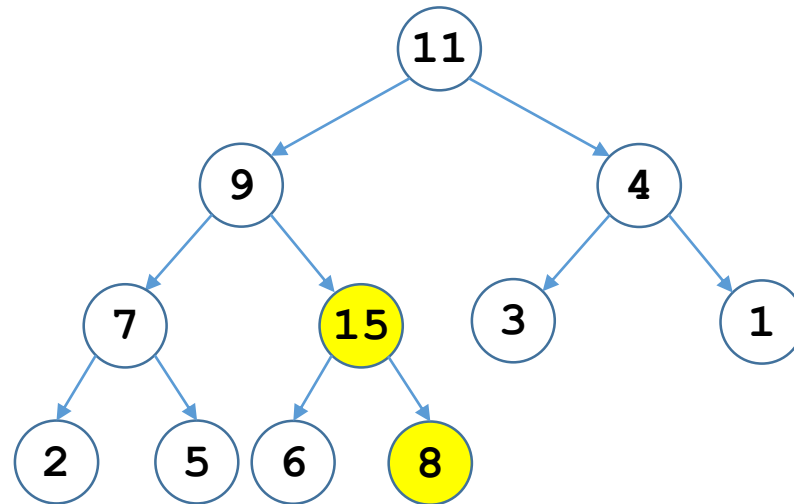
Enqueue 15

Index	0	1	2	3	4	5	6	7	8	9	10
value	11	9	4	7	8	3	1	2	5	6	15

Length = 11

The **Enqueue** operation

- Increment **length**
- Add the item to be enqueued as the last leaf node (at index **length-1**)
- Perform **ReheapUp** operation



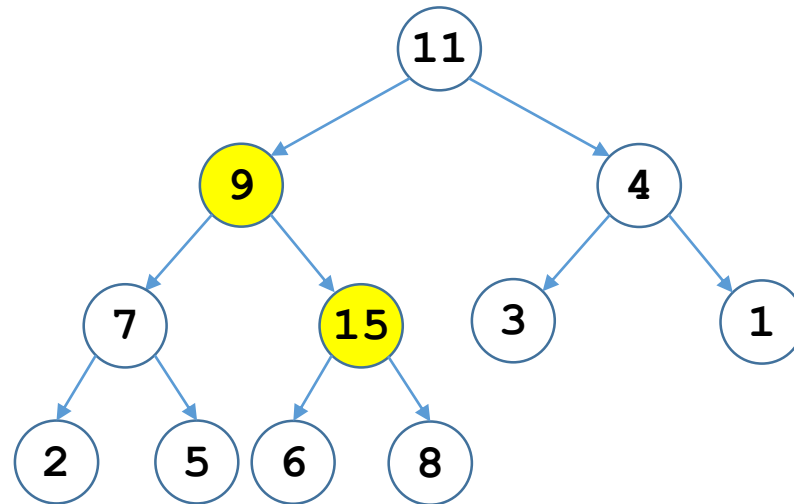
Enqueue 15

Index	0	1	2	3	4	5	6	7	8	9	10
value	11	9	4	7	15	3	1	2	5	6	8

Length = 11

The **Enqueue** operation

- Increment **length**
- Add the item to be enqueued as the last leaf node (at index **length-1**)
- Perform **ReheapUp** operation



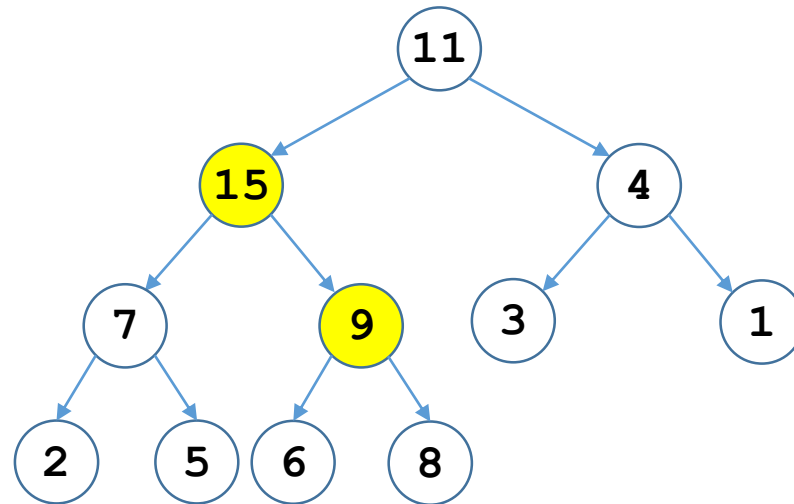
Enqueue 15

Index	0	1	2	3	4	5	6	7	8	9	10
value	11	9	4	7	15	3	1	2	5	6	8

Length = 11

The **Enqueue** operation

- Increment **length**
- Add the item to be enqueued as the last leaf node (at index **length-1**)
- Perform **ReheapUp** operation



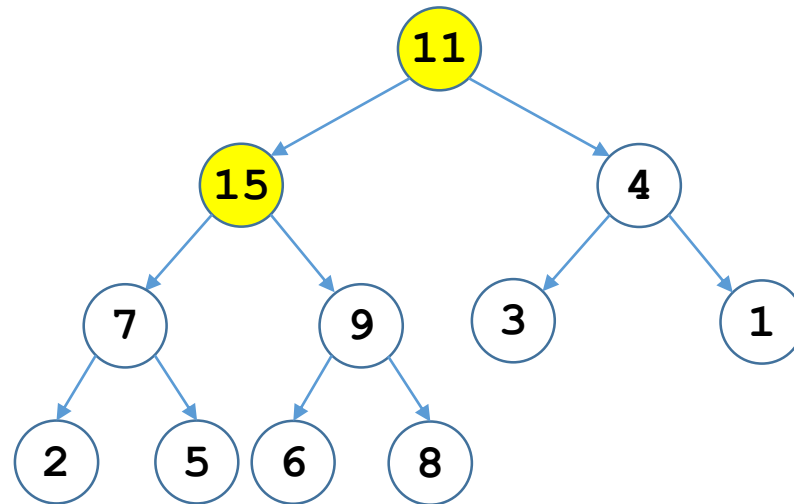
Enqueue 15

Index	0	1	2	3	4	5	6	7	8	9	10
value	11	15	4	7	9	3	1	2	5	6	8

Length = 11

The **Enqueue** operation

- Increment **length**
- Add the item to be enqueued as the last leaf node (at index **length-1**)
- Perform **ReheapUp** operation



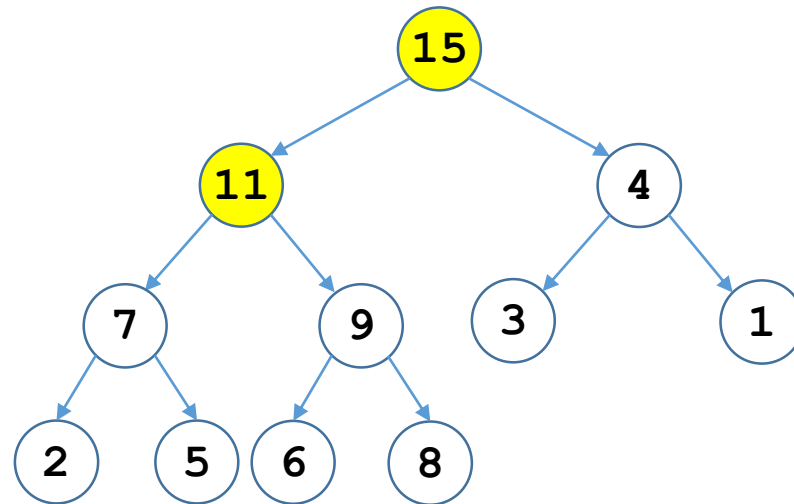
Enqueue 15

Index	0	1	2	3	4	5	6	7	8	9	10
value	11	15	4	7	9	3	1	2	5	6	8

Length = 11

The **Enqueue** operation

- Increment **length**
- Add the item to be enqueued as the last leaf node (at index **length-1**)
- Perform **ReheapUp** operation



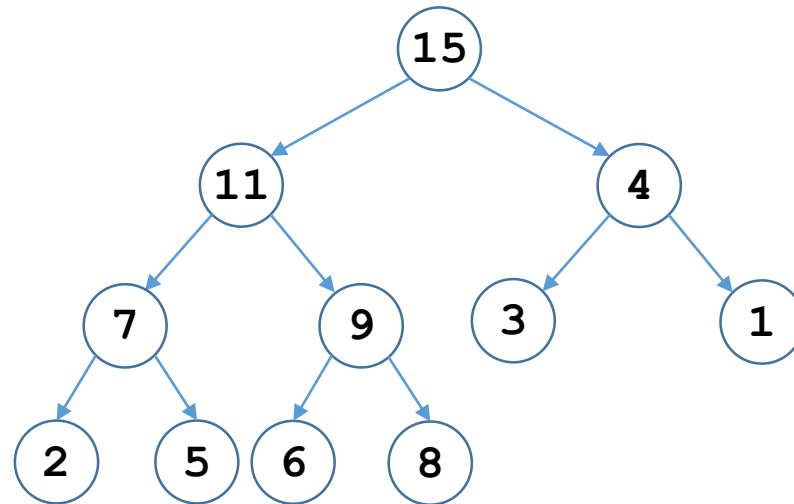
Enqueue 15

Index	0	1	2	3	4	5	6	7	8	9	10
value	15	11	4	7	9	3	1	2	5	6	8

Length = 11

The **Enqueue** operation

- Increment **length**
- Add the item to be enqueued as the last leaf node (at index **length-1**)
- Perform **ReheapUp** operation



Enqueue 15

Index	0	1	2	3	4	5	6	7	8	9	10
value	15	11	4	7	9	3	1	2	5	6	8

Length = 11

pqtype.cpp

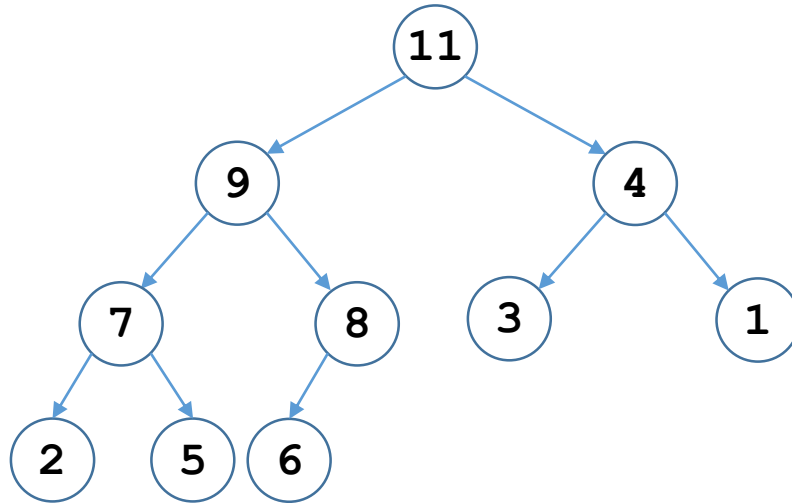
```
template<class ItemType>
void PQType<ItemType>::Enqueue(ItemType newItem)
{
    if (length == maxItems)
        throw FullPQ();
    else
    {
        length++;
        items.elements[length-1] = newItem;
        items.ReheapUp(0, length-1);
    }
}
```

pqtype.cpp

```
template<class ItemType>
void PQType<ItemType>::Enqueue(ItemType newItem)
{
    if (length == maxItems)
        throw FullPQ();
    else
    {
        length++;
        items.elements[length-1] = newItem;
        items.ReheapUp(0, length-1);
    }
}
```

$O(\log N)$

The **Dequeue** operation

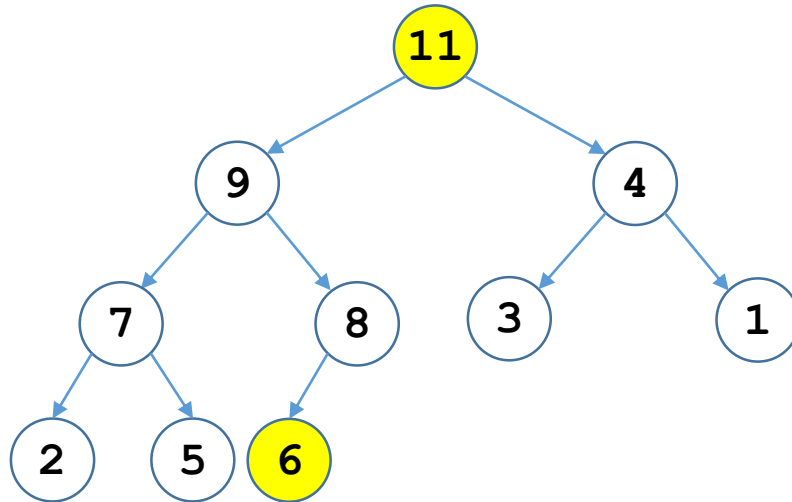


Index	0	1	2	3	4	5	6	7	8	9
value	11	9	4	7	8	3	1	2	5	6

Length = 10

The **Dequeue** operation

- Replace the root with the last leaf node



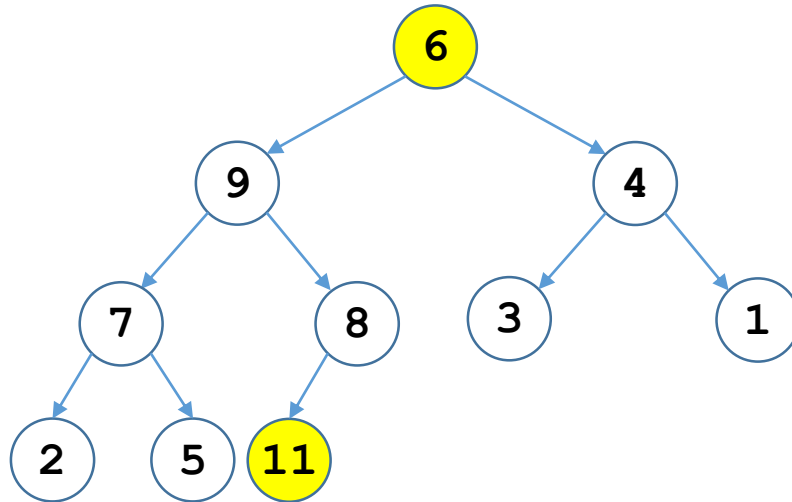
Index	0	1	2	3	4	5	6	7	8	9
value	11	9	4	7	8	3	1	2	5	6

Length = 10



The **Dequeue** operation

- Replace the root with the last leaf node



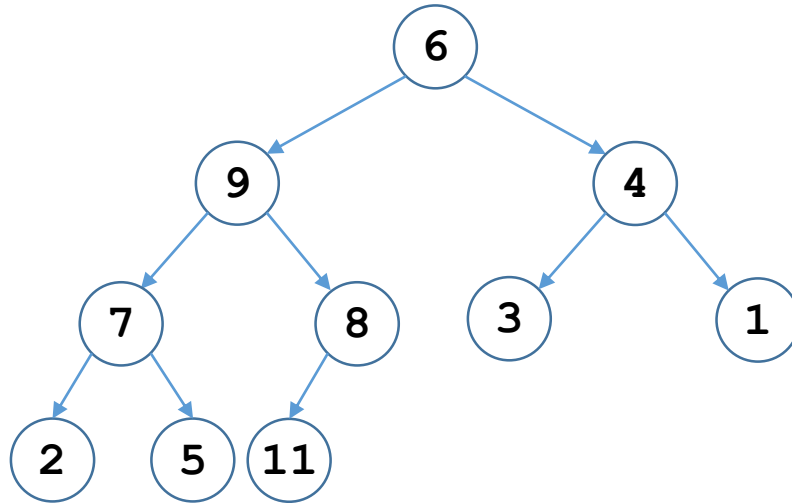
Index	0	1	2	3	4	5	6	7	8	9
value	6	9	4	7	8	3	1	2	5	11

Length = 10



The **Dequeue** operation

- Replace the root with the last leaf node

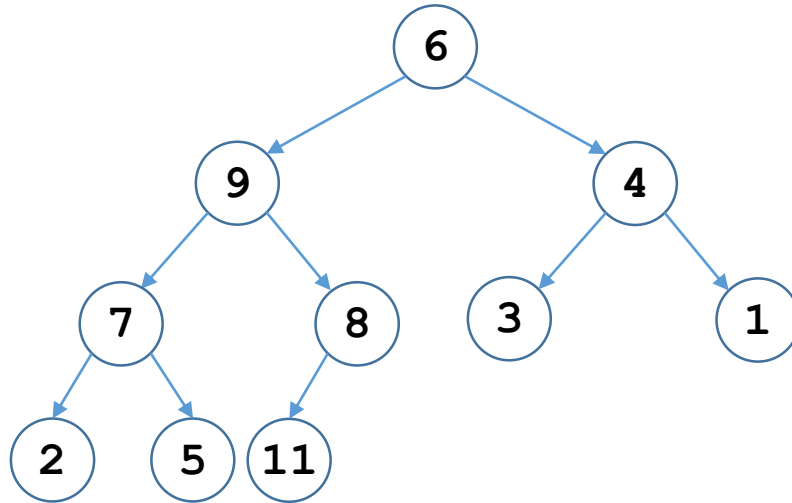


Index	0	1	2	3	4	5	6	7	8	9
value	6	9	4	7	8	3	1	2	5	11

Length = 10

The **Dequeue** operation

- Replace the root with the last leaf node
- Decrement **length** (the last leaf node is out of the tree now)

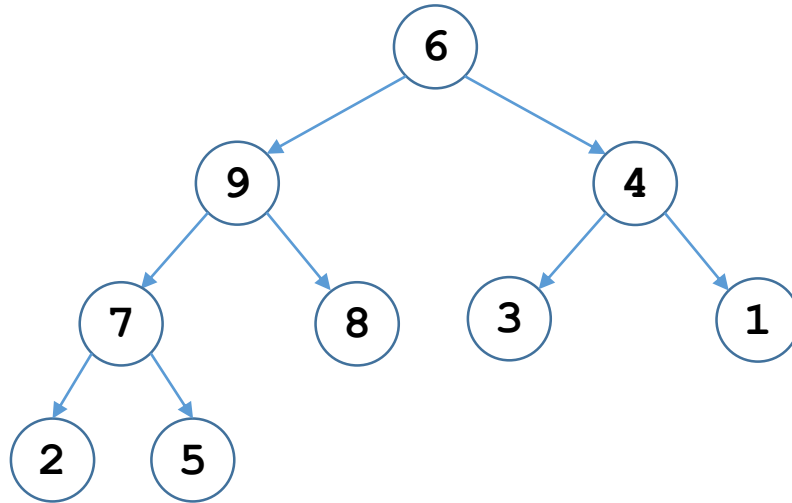


Index	0	1	2	3	4	5	6	7	8	9
value	6	9	4	7	8	3	1	2	5	11

Length = 10

The **Dequeue** operation

- Replace the root with the last leaf node
- Decrement **length** (the last leaf node is out of the tree now)

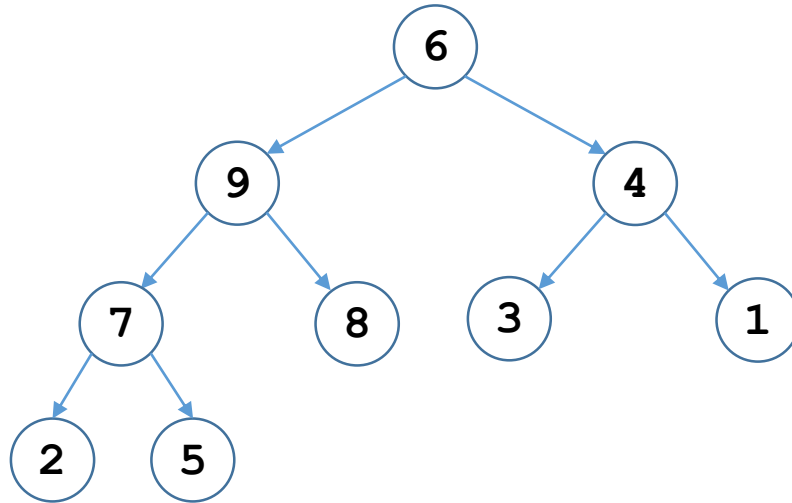


Index	0	1	2	3	4	5	6	7	8	9
value	6	9	4	7	8	3	1	2	5	11

Length = 9

The **Dequeue** operation

- Replace the root with the last leaf node
- Decrement **length** (the last leaf node is out of the tree now)
- Perform **ReheapDown** operation

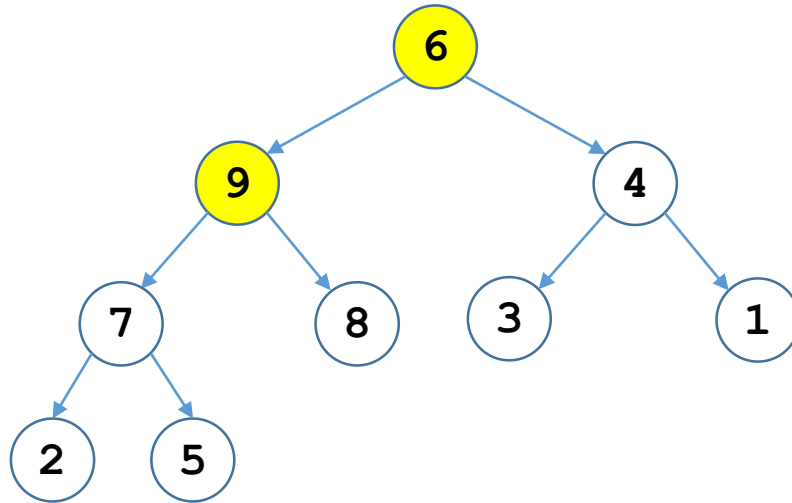


Index	0	1	2	3	4	5	6	7	8	9
value	6	9	4	7	8	3	1	2	5	11

Length = 9

The **Dequeue** operation

- Replace the root with the last leaf node
- Decrement **length** (the last leaf node is out of the tree now)
- Perform **ReheapDown** operation

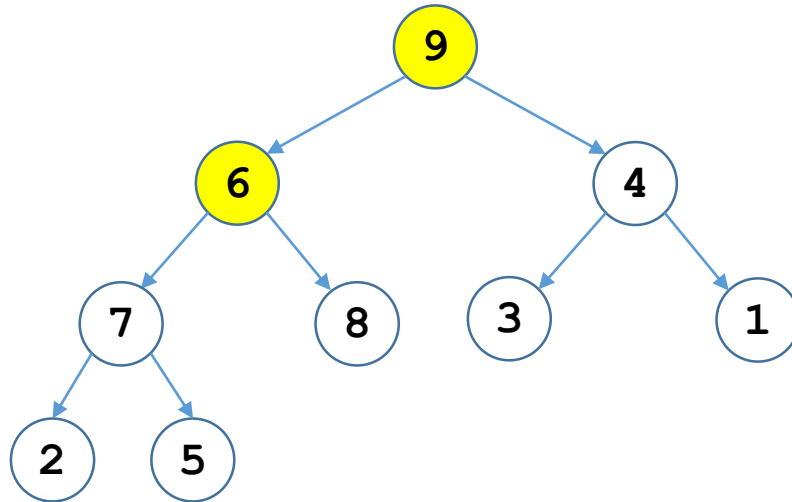


Index	0	1	2	3	4	5	6	7	8	9
value	6	9	4	7	8	3	1	2	5	11

Length = 9

The **Dequeue** operation

- Replace the root with the last leaf node
- Decrement **length** (the last leaf node is out of the tree now)
- Perform **ReheapDown** operation

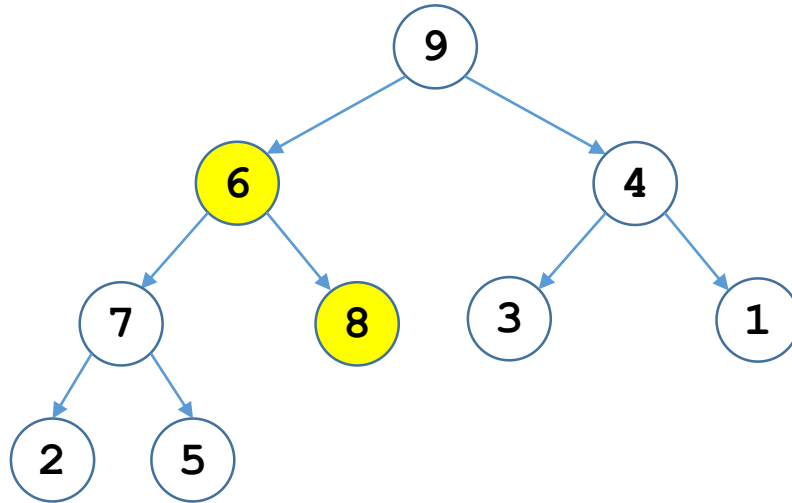


Index	0	1	2	3	4	5	6	7	8	9
value	9	6	4	7	8	3	1	2	5	11

Length = 9

The **Dequeue** operation

- Replace the root with the last leaf node
- Decrement **length** (the last leaf node is out of the tree now)
- Perform **ReheapDown** operation

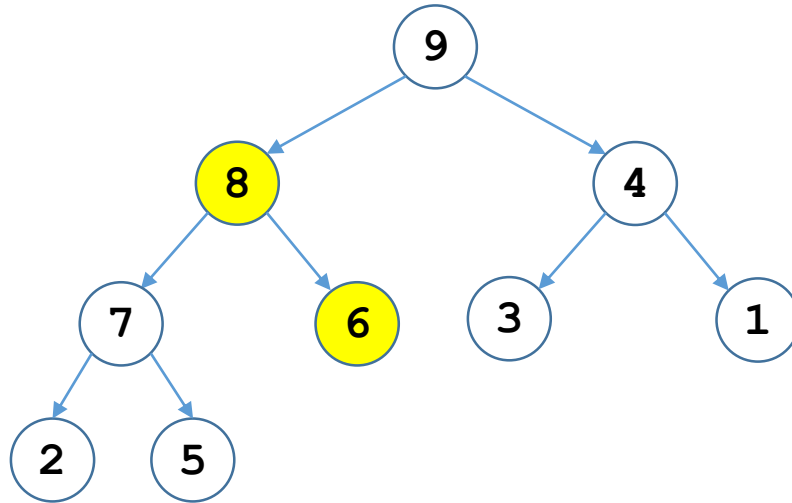


Index	0	1	2	3	4	5	6	7	8	9
value	9	6	4	7	8	3	1	2	5	11

Length = 9

The **Dequeue** operation

- Replace the root with the last leaf node
- Decrement **length** (the last leaf node is out of the tree now)
- Perform **ReheapDown** operation

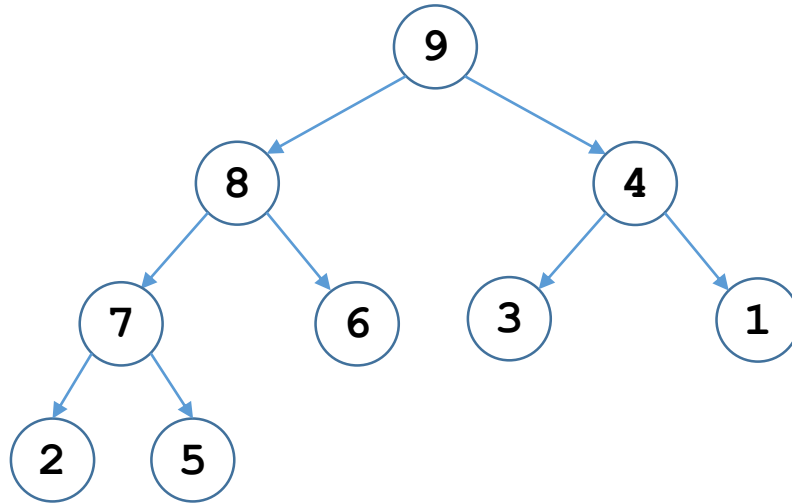


Index	0	1	2	3	4	5	6	7	8	9
value	9	8	4	7	6	3	1	2	5	11

Length = 9

The **Dequeue** operation

- Replace the root with the last leaf node
- Decrement **length** (the last leaf node is out of the tree now)
- Perform **ReheapDown** operation



Index	0	1	2	3	4	5	6	7	8	9
value	9	8	4	7	6	3	1	2	5	11

Length = 9

pqtype.cpp

```
template<class ItemType>
void PQType<ItemType>::Dequeue(ItemType& item)
{
    if (length == 0)
        throw EmptyPQ();
    else
    {
        item = items.elements[0];
        items.elements[0] = items.elements[length-1];
        length--;
        items.ReheapDown(0, length-1);
    }
}
```

pqtype.cpp

```
template<class ItemType>
void PQType<ItemType>::Dequeue(ItemType& item)
{
    if (length == 0)
        throw EmptyPQ();
    else
    {
        item = items.elements[0];
        items.elements[0] = items.elements[length-1];
        length--;
        items.ReheapDown(0, length-1);
    }
}
```

$O(\log N)$